

Supplementary I (Forking Lemma)

In the manuscript, we claim that the analysed signature schemes adopt improper employment of the forking lemma in their security proofs. They all employ the original general forking lemma.

I. GENERAL FORKING LEMMA [1]

The original general forking lemma was proposed by Pointcheval et al. for security proofs, especially for pairing-free signature schemes. We briefly describe the general forking lemma and explain why it cannot guarantee security in the aggregate scenarios.

Theorem I.1 (Forking Lemma). *Let $\mathcal{A}$ be a probabilistic polynomial time (PPT) algorithm whose input only consists of public data. Assume that $\mathcal{A}$ produces a valid signature (m, u, c, z) with a non-negligible probability ϵ . Then there is another algorithm $\mathcal{C}$ which has control over $\mathcal{A}$ and produces two valid signatures (m, u, c, z) and (m, u, c', z') such that $h \neq h'$ with another non-negligible probability $\frac{\epsilon^2}{Q_h}$ (where Q_h is the total number of random oracle queries made by the adversary).*

In the theorem, Pointcheval et al. did not explicitly state a prerequisite regarding the input format of the random oracle. However, in their formal proof, they require that $\mathcal{A}$ queries the random oracle $\mathcal{H}$ on the pair (m, u) Q_h times.

“Finally, after less than $2/\epsilon + 14Q_h/\epsilon$ repetitions of the attack, with probability greater than $\frac{1}{5} \times \frac{3}{5} \geq \frac{1}{9}$, we have obtained two valid signatures (m, u, c, z) and (m', u', c', z') with $(m, u) = (m', u')$ and distinct challenges $c = \mathcal{H}(m, u) \neq \mathcal{H}(m', u') = c'$.”

In short, (m, u) must be fed as a single input to $\mathcal{H}$. In aggregate signature schemes, a valid aggregate signature contains $(\{m_i, u_i\}_{i=1}^n, \{c_i\}_{i=1}^n, z)$, but $\{m_i, u_i\}_{i=1}^n$ are never queried as a single input to $\mathcal{H}$. Therefore, those pairing-free CLAS schemes cannot invoke the forking-lemma guarantee, and their security proofs are inapplicable.

II. CHOSEN-MESSAGE ATTACK IN AGGREGATE SIGNATURE SCHEMES [2]

Many previous works claimed that the security of an aggregate signature is equivalent to that of its individual constituent signatures. However, Boneh et al. [2] formalized a dedicated security model for aggregate signatures, which differs from the standard individual signature model, since an adversary may forge an aggregate signature without forging any individual signature.

We adopt the aggregate signature security model verbatim from Subsection 3.2 of Boneh et al.’s paper. Unlike the chosen-message attack model used in traditional signature schemes, the aggregate signature security model is a chosen-key and chosen-message attack model. In this model, adversary $\mathcal{A}$ can

3.2 Aggregate Signature Security

Informally, the security of aggregate signature schemes is equivalent to the nonexistence of an adversary capable, within the confines of a certain game, of existentially forging an aggregate signature. Existential forgery here means that the adversary attempts to forge an aggregate signature, on messages of his choice, by some set of users.

We formalize this intuition as the aggregate chosen-key security model. In this model, the adversary $\mathcal{A}$ is given a single public key. His goal is the existential forgery of an aggregate signature. We give the adversary power to choose all public keys except the challenge public key. The adversary is also given access to a signing oracle on the challenge key. His advantage, $\text{Adv}_{\text{AggSig}, \mathcal{A}}$, is defined to be his probability of success in the following game.

Setup. The aggregate forger $\mathcal{A}$ is provided with a public key PK_1 , generated at random.

Queries. Proceeding adaptively, $\mathcal{A}$ requests signatures with PK_1 on messages of his choice.

Response. Finally, $\mathcal{A}$ outputs $k - 1$ additional public keys $PK_2, \dots, PK_k$. Here k is at most N , a game parameter. These keys, along with the initial key PK_1 , will be included in $\mathcal{A}$ ’s forged aggregate. $\mathcal{A}$ also outputs messages $M_1, \dots, M_k$; and, finally, an aggregate signature σ by the k users, each on his corresponding message.

not only choose messages to query the signing oracle, but also can choose all public keys other than the challenge public key. Although the original general forking lemma has been widely applied and further generalized to prove the security of numerous digital signature schemes and other random-oracle-based cryptographic primitives, this does not imply its validity in aggregate signature settings.

III. FORMAL PROOF

According to the aggregate signature security model of Boneh, there are two scenarios for forging an aggregate signature.

- In the first scenario, $\mathcal{A}$ -I forges an individual signature and aggregates it. The general forking lemma is valid under $\mathcal{A}$ -I scenario.
- In the second scenario, $\mathcal{A}$ -II cannot forge any individual signature, yet can forge an aggregate signature.

We prove that a secure aggregate signature scheme against $\mathcal{A}$ -II cannot invoke the forking lemma guarantee.

Theorem III.1. *No secure aggregate signature scheme against $\mathcal{A}$ -II can be proven under the original general forking lemma (Theorem I.1) in the random oracle model.*

Proof. First, we formalize the core guarantee of the general forking lemma: if a PPT adversary can output a valid signature forgery with non-negligible probability ϵ in the random oracle model, then there exists a PPT replaying algorithm that can reset the adversary’s random tape, keep all random oracle responses unchanged except for one challenge random query, and output two valid signatures (m, u, c, z) and (m, u, c', z') , adversaries with non-negligible probability $\frac{\epsilon^2}{Q_h}$ (where Q_h is the total number of random oracle queries made by the adversary).

If $\mathcal{A}$ -II is able to forge two aggregate signatures $(\{u_i\}_{i=1}^n, z)$ and $(\{u_i\}_{i=1}^n, z')$ according to the general forking lemma, then we can construct another algorithm $\mathcal{B}$ which can forge an individual signature.

$\mathcal{A}$ -II can output a valid aggregate signature $(\{u_i\}_{i=1}^n, z)$ with non-negligible probability ϵ . According to the general

forking lemma, $\mathcal{B}$ replays the same random tape, and obtains two valid aggregate signatures $(\{u_i\}_{i=1}^n, z)$ and $(\{u_i\}_{i=1}^n, z')$ with non-negligible probability. We now derive the rigid constraints from the replay rules:

- **Fixed commitment set:** The random tape of $\mathcal{A}$ -II is identical across the two executions, so the random values $\{r_i\}_{i=1}^n$ are fixed, which means the commitment set $\{u_i = r_i P\}_{i=1}^n$ must be exactly the same in both signatures.
- **Fixed non-challenge random oracle responses:** According to the general forking lemma rules, all oracle queries are unchanged except for the challenge query $c_1 = \mathcal{H}(u_1, y_1)$. Thus, the hash challenges $\{c_i\}_{i=2}^n$ are fixed.
- **Fixed key pairs:** The public keys $\{y_i\}_{i=1}^n$ must be fixed as $\mathcal{A}$ -II's public key selection is determined by its fixed random tape, so $\{x_i\}_{i=1}^n$ are fixed.
- **Fixed non-challenge signature response:** For $i \in [2, n]$, $z_i = \text{Sign}(r_i, c_i, x_i)$. Since r_i, c_i, x_i are all fixed, the set $\{z_i\}_{i=2}^n$ is identical across the two executions.

Finally $\mathcal{B}$ generates $(\{u_i\}_{i=1}^n, z')$ according to the general forking lemma, where both $(\{u_i\}_{i=1}^n, z)$ and $(\{u_i\}_{i=1}^n, z')$ pass the aggregate-verification algorithm, where $z = z_1 + \sum_{i=2}^n z_i$ and $z' = z'_1 + \sum_{i=2}^n z_i$. We can combine the two equations to solve for the private key x_1 of the challenge signer:

$$z - z' = z_1 - z'_1$$

If x_1 can be extracted from $(z_1 - z'_1, c_1, c'_1)$, then $\mathcal{B}$ can extract x_1 from $(z - z', c_1, c'_1)$.

This means algorithm $\mathcal{B}$ can use the two aggregate signatures generated via the general forking lemma to extract the private key x_1 , and thus forge a valid individual signature for the challenge public key y_1 with non-negligible probability. This directly contradicts the core definition of $\mathcal{A}$ -II, which cannot forge any valid individual signature for the challenge public key. $\square$

IV. EXAMPLE

We take Schnorr signature as an example. Schnorr signature scheme is an non-interactive proof of knowledge of discrete log.

$$S(P, x, y = xP) :$$

$$r \rightarrow \mathbb{Z}_p;$$

$$u = rP;$$

$$c = \mathcal{H}(y, u);$$

$$z = r + cx;$$

$$\text{output} : (u, z);$$

$$V(P, y, (u, z))$$

$$c = \mathcal{H}(y, u);$$

$$zP \stackrel{?}{=} u + cy$$

If Schnorr signatures $(\{u_i, z_i\}_{i=1}^n)$ are aggregated as $(\{u_i\}_{i=1}^n, z = \sum_{i=1}^n z_i)$, the aggregate verification algorithm is:

$$A - V(P, \{y_i, u_i\}_{i=1}^n, z)$$

$$c_i = \mathcal{H}(y_i, u_i);$$

$$zP \stackrel{?}{=} \sum_{i=1}^n u_i + \sum_{i=1}^n c_i y_i$$

The signer knows x_1 from $y_1 = x_1 P$.

An adversary $\mathcal{A}$ does not know x_1 , but has x_2 from $y_2 = x_2 P$. $\mathcal{A}$ randomly generates u_1 , and computes $c_1 = \mathcal{H}(y_1, u_1)$ and $u_2 = -u_1 - c_1 y_1$, $c_2 = \mathcal{H}(y_2, u_2)$, and $z = c_2 x_2$, and sets (u_1, u_2, z) as the aggregate signature. The signature is valid since

$$u_1 + u_2 + c_1 y_1 + c_2 y_2$$

$$= -u_1 - u_1 - c_1 y_1 + c_1 y_1 + c_2 y_2$$

$$= c_2 y_2$$

$$= c_2 x_2 P$$

$$= zP$$

V. APPLICATIONS OF THE FORKING LEMMA [3,4]

The original general forking lemma is not inherently invalid for aggregate signatures. Rather, existing certificateless aggregate signature schemes misapply the forking lemma because the general forking lemma does not satisfy its necessary prerequisites. In fact, Maxwell et al. [3] and Chen et al. [4] extended the general forking lemma to construct provably secure pairing-free PKI-based aggregate signature schemes. However, Maxwell's scheme is not an incremental aggregate signature; Chen's scheme is a sequential aggregate signature. They cannot meet the incremental aggregation property of Boneh's aggregate signature scheme [2].

In Section VI.B of our manuscript, we emphasize that constructing secure aggregate signature schemes that correctly satisfy the requirements of the forking lemma is an important future research direction, which indicates that the forking lemma remains valid and applicable when used properly.

References:

- [1] D.Pointcheval and J. Stern, "Security arguments for digital signatures and blind signatures," in *Journal of Cryptology*, vol. 13, no. 3, pp. 361–396, 2000.
- [2] D. Boneh, C. Gentry, H. Shacham, and B. Lynn, "Aggregate and Verifiably Encrypted Signatures from Bilinear Maps," In *proceedings of Eurocrypt 2003*, LNCS 2656, pp. 416–432, 2003
- [3] G. Maxwell, A. Poelstra, Y. Seurin, Y. "Simple Schnorr multi-signatures with applications to Bitcoin", in *Des. Codes Cryptogr.*, vol. 87, pp. 2139–2164, 2019.
- [4] Y. Chen, Y. Zhao, "Half-Aggregation of Schnorr Signatures with Tight Reductions", in *Computer Security – ESORICS 2022. ESORICS 2022, Lecture Notes in Computer Science*, vol 13555. 2022.